

S10 - Avance Consolidado

Patricio Bayas Meza - Jose Puebla Paladines - Marco Zurita Rojas

2026-08-23

https://github.com/marcoz1691/MIA_Deteccion_HC

1. Resumen ejecutivo

Este informe consolida el avance del proyecto MIA_Deteccion_HC: deteccion de inconsistencias en historias clinicas a nivel de oracion, con evaluacion sobre MEDEC (Ben Abacha et al., 2025) y prototipo demo para CITIMED.

Resultado principal: reformular la tarea de nivel nota a nivel oracion eleva el ROC-AUC de 0.504 (baseline) a 0.949 (IC95 0.940-0.958) y localiza la oracion erronea en 84.6 % de las notas con error (263/311).

AUPRC test = 0.419 con prevalencia de oracion-error 4.5 % - metrica complementaria al F1 en clases desbalanceadas.

Se implementaron tres brazos (TF-IDF, LLM zero-shot, LLM+RAG), demo Streamlit, API REST FastAPI (POST /generar), frontend React, fallback explicito a TF-IDF, soporte Ollama on-premise para PHI y evaluaciones por tipo de error y sesgo EN-ES.

Repositorio: https://github.com/marcoz1691/MIA_Deteccion_HC

Disclaimer: prototipo de investigacion; no sustituye criterio clinico ni decisiones terapeuticas.

2. Metodologia

2.1 Unidad de prediccion: oracion clinica segmentada desde la nota completa (Error Sentence ID en MEDEC). Localizacion = oracion con mayor score dentro de la nota.

2.2 Stack implementado (alineado con el repositorio):

- TF-IDF + rasgos numericos + Regresion Logistica (s6/modelo_ajustado.py)
- LLM zero-shot via cliente OpenAI-compatible (s7/llm_client.py, s7/prompts.py)
- RAG: sentence-transformers + FAISS sobre guias en s7/knowledge/ (s7/rag_index.py)
- Inferencia unificada: s7/inferencia.py (demo, API y scripts de evaluacion)
- API REST: FastAPI en api/ - GET /health, POST /generar (InferenceService)
- Frontend React + Vite en frontend/ - proxy a la API en puerto 8000
- Config centralizada: s7/config.yaml; seed global SEED=42

2.3 Trabajo futuro (NO implementado en S10): LangChain/LlamaIndex, Docker Compose, DVC, MLflow operativo (requirements-optional.txt), cascada TF-IDF->LLM en inferencia, correccion sugerida/faithfulness RAG, indice CIE-10 formal, corpus CITIMED anonimizado.

3. Implementacion - capacidades entregadas

La tabla siguiente corrige el borrador S8 (?132-148): todas estas capacidades estan implementadas en el repositorio con rutas verificables.

Capacidad	Modulo	Evidencia
Comparacion tripartita TF-IDF/LLM/LLM+RAG	s7/eval_tripartita.py	salidas_s7/metricas_tripartita.json
Indice FAISS + RAG	s7/rag_index.py, s7/knowledge/	salidas_s7/faiss_index/
Inferencia unificada	s7/inferencia.py	s7/test_fallback.py
Analisis por ErrorType	s7/analisis_por_tipo.py	salidas_s7/analisis_por_tipo.json
Sesgo EN-ES	s7/eval_idioma.py, s7/eval_tfidf_idioma.py	salidas_s7/eval_idioma_en_es.json
Demo Streamlit	demo/	demo/README.md + capturas s10/evidencias/capturas/
API REST FastAPI	api/main.py, api/service.py	GET /health, POST /generar, OpenAPI /docs
Frontend React	frontend/	NotalInput + ResultadosPanel, proxy Vite -> :8000

Fallback API -> TF-IDF	s7/inferencia.py, api/service.py	modo_degradado=True, test_fallback
SHAP/LIME (figuras)	salidas_s7/shap_*.png, lime_*.png	s7/docs/reporte_interpretabilidad_etica.md
Mock LLM + consentimiento PHI + auditoria SHA-256	demo/components/security.py	salidas_s7/audit.log
Ollama on-premise (PHI CITIMED)	s7/llm_client.py, s7/docs/informe_produccion.md	
 OPENAI_BASE_URL=http://localhost:11434/v1 |

4. Resultados TF-IDF ajustado (test MEDEC)

ROC-AUC = 0.949 (IC95 0.940-0.958)

AUPRC = 0.419 (IC95 0.392-0.446)

F1 = 0.497, Precision = 0.357, Recall = 0.817

CV 5-fold AUC = 0.965 +/- 0.007

Localizacion top-1 = 84.6 % (263/311 notas)

Baseline a nivel nota: ROC-AUC = 0.504 (sin senal lexica). McNemar baseline vs ajustado a nivel nota: p = 0.53 - el valor esta en la localizacion, no en el flag binario por nota.

Figuras embebidas a continuacion (origen: s6/figura_ajuste.png y s10/evidencias/).

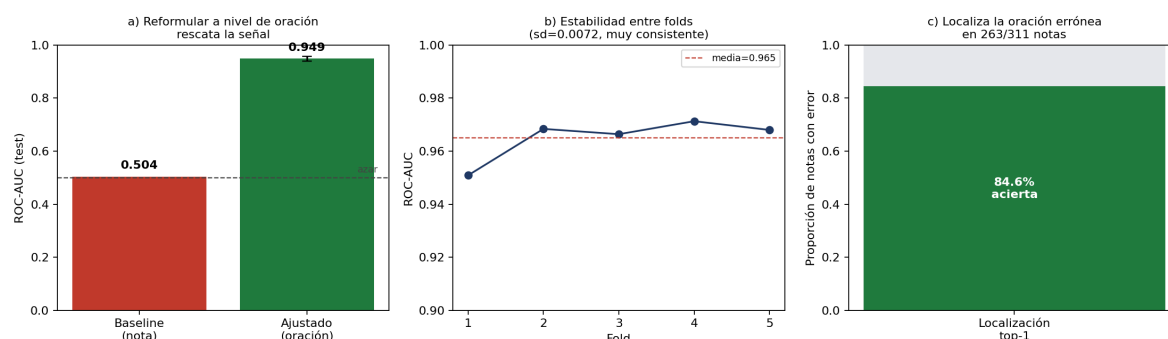


Figura 1. Curvas ROC/PR y lift - TF-IDF ajustado (MEDEC).

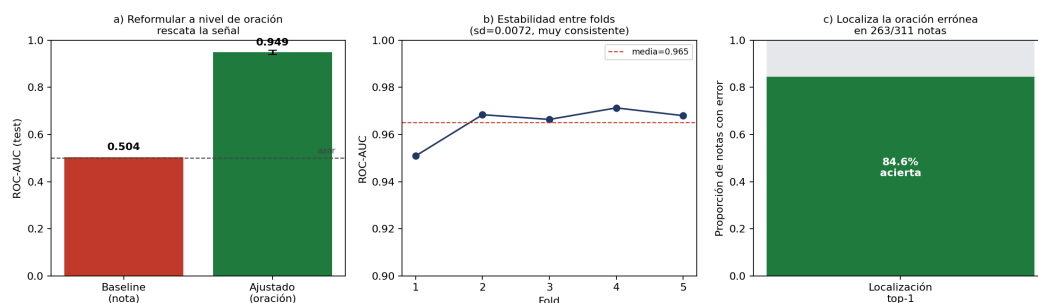


Figura 1b. Curvas TF-IDF (evidencia S6).

5. Resultados S7 extendido

5.1 Comparacion tripartita (subset test, mock LLM declarado como limitacion):

- TF-IDF: ROC-AUC = 0.954, AUPRC = 0.475, latencia ~ 0.41 ms/oracion
- LLM zero-shot: ROC-AUC = 0.498 (mock)
- LLM+RAG: ROC-AUC = 0.498

En modo mock, LLM no discrimina (AUC ~ 0.5); TF-IDF mantiene ventaja estadistica (McNemar TF-IDF vs LLM: p ~ 0.11 en subset 500 oraciones). Evaluacion con API real pendiente para latencias y costos de produccion.

5.2 Analisis por ErrorType (TF-IDF, test):

causalOrganism: recall=0.818, loc top-1=0.818 (n=11)

diagnosis: recall=0.897, loc top-1=0.948 (n=116)

management: recall=0.680, loc top-1=0.732 (n=97)

pharmacotherapy: recall=0.861, loc top-1=0.861 (n=36)
treatment: recall=0.863, loc top-1=0.824 (n=51)
Errores criticos (pharmacotherapy, diagnosis) mantienen recall > 0.86.

5.3 Sesgo EN-ES (subset n=200, mock LLM): Delta AUC EN?ES = 0.000. Sin ventaja medible en mock; para CITIMED se recomienda prompts en español + modelo local (Ollama).

5.4 TF-IDF por stop_words (test MEDEC, eval_tfidf_idioma.py):

- english: AUC=0.949, AUPRC=0.398
- spanish: AUC=0.959, AUPRC=0.432
- bilingue (none): AUC=0.959, AUPRC=0.432

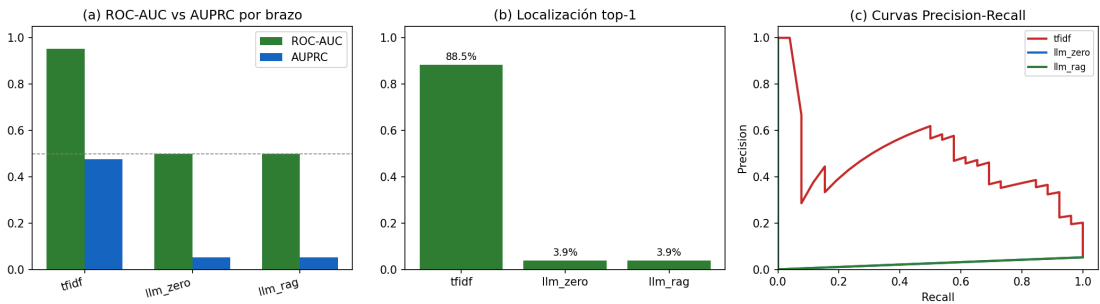


Figura 2. Comparacion tripartita TF-IDF / LLM / LLM+RAG.

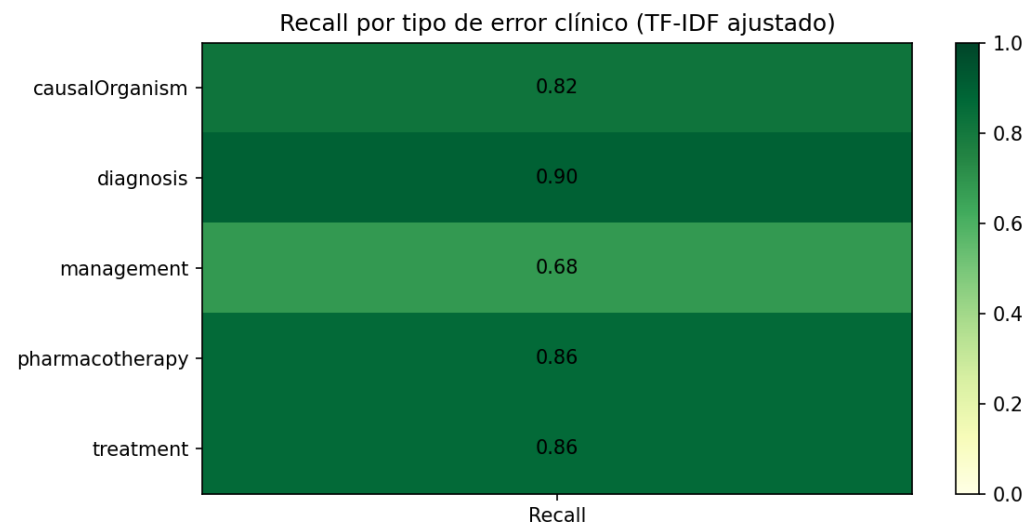


Figura 3. Recall por ErrorType (TF-IDF).

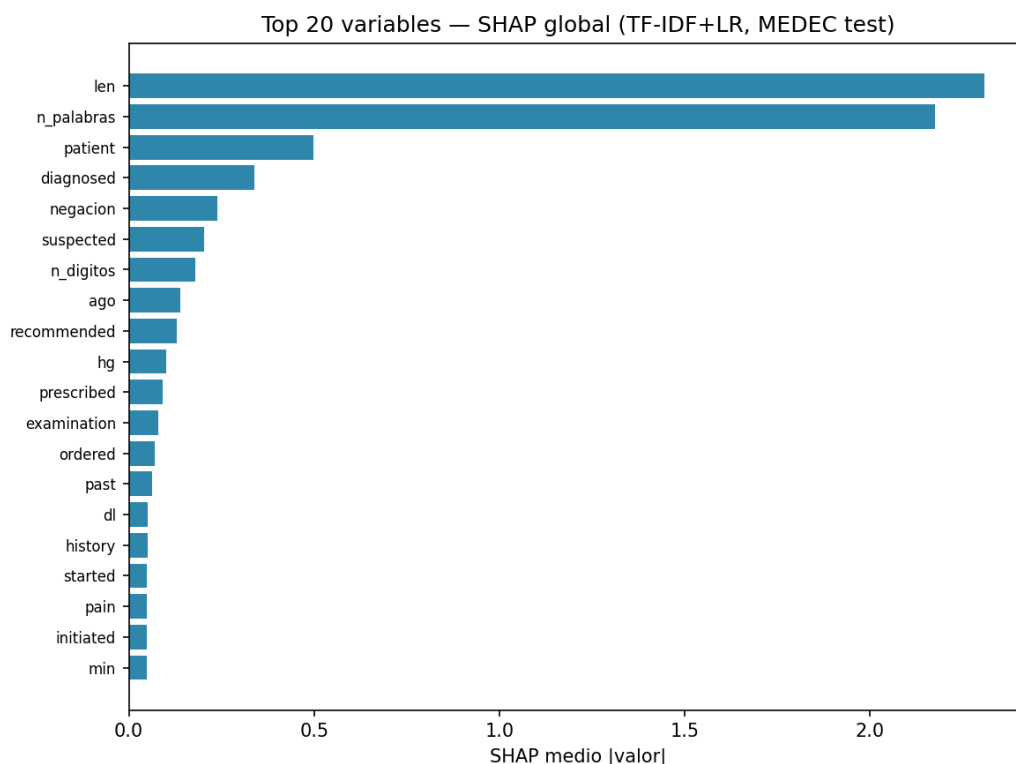


Figura 4. Interpretabilidad SHAP (TF-IDF).

6. Demo, API y robustez operativa

6.1 Demo Streamlit (demo/app.py): analisis tripartita, pagina Metricas, configuracion de brazos.

6.2 API FastAPI (api/): expone el mismo pipeline que la demo via HTTP.

- GET /health - estado del servicio y disponibilidad del modelo TF-IDF
- POST /generar - payload JSON: nota_clinica, mock_llm (default true), idioma, brazos, umbral
- InferenceService carga TF-IDF, LLMClient y RAGIndex al arrancar (lifespan)
- CORS habilitado para el frontend React; documentacion OpenAPI en /docs
- Arranque: uvicorn api.main:app --reload --port 8000

6.3 Frontend React (frontend/): cliente minimo con Vite.

- npm run dev -> http://localhost:5173; proxy /generar y /health hacia :8000
- Muestra top1 (oracion sospechosa) y tabla de scores por brazo

6.4 Seguridad y fallback:

Mock LLM activo por defecto - ningun dato sale del equipo sin consentimiento explicito (Streamlit).

Fallback: si la API LLM falla (LLMUnavailableError), inferencia continua con TF-IDF, modo_degradado=True y mensaje en la respuesta JSON (verificado en s7/test_fallback.py).

Ollama on-premise: mismo LLMClient con OPENAI_BASE_URL=http://localhost:11434/v1 para datos PHI CITIMED sin egress a nube.

Auditoria Streamlit: salidas_s7/audit.log registra hash SHA-256 de la nota - sin texto clinico.

Capturas del prototipo embebidas a continuacion (s10/evidencias/capturas/).

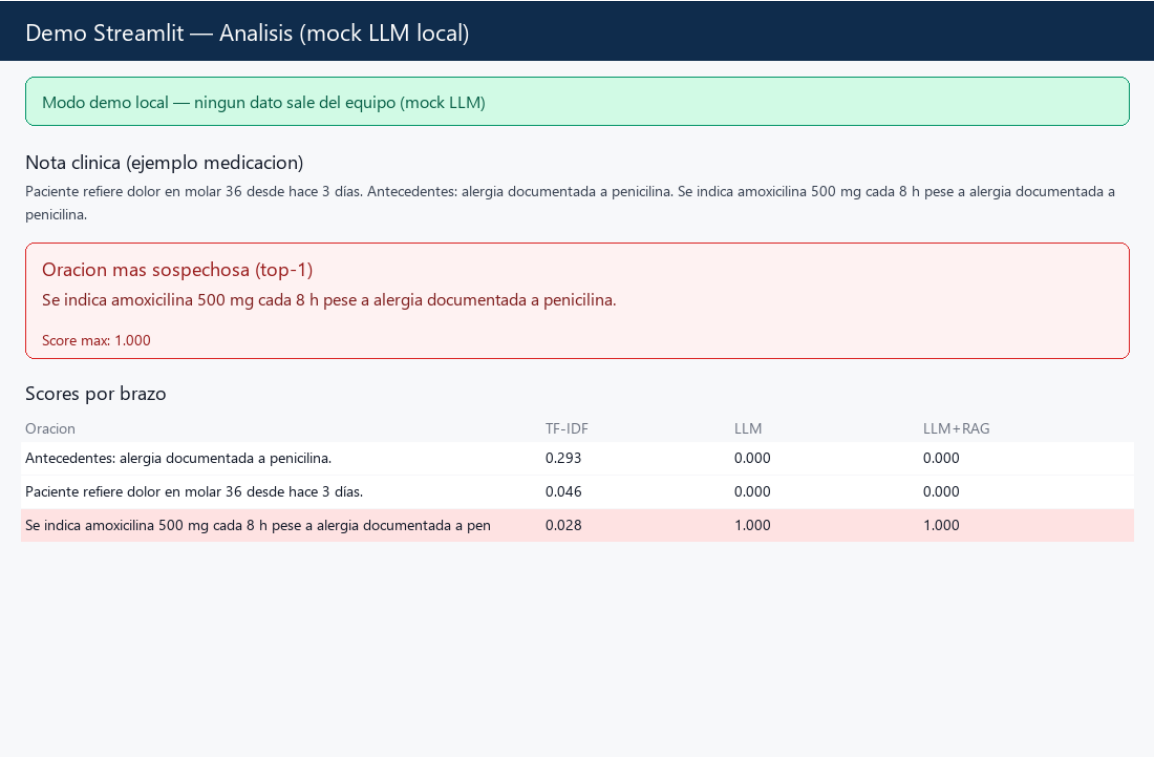


Figura 5. Demo - analisis ejemplo medicacion (mock LLM).

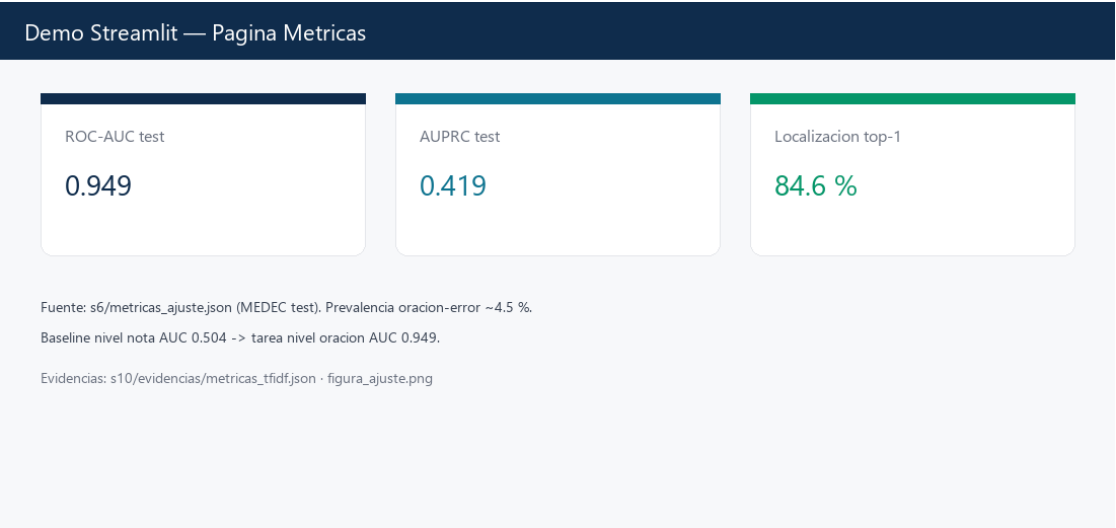


Figura 6. Demo - pagina Metricas.

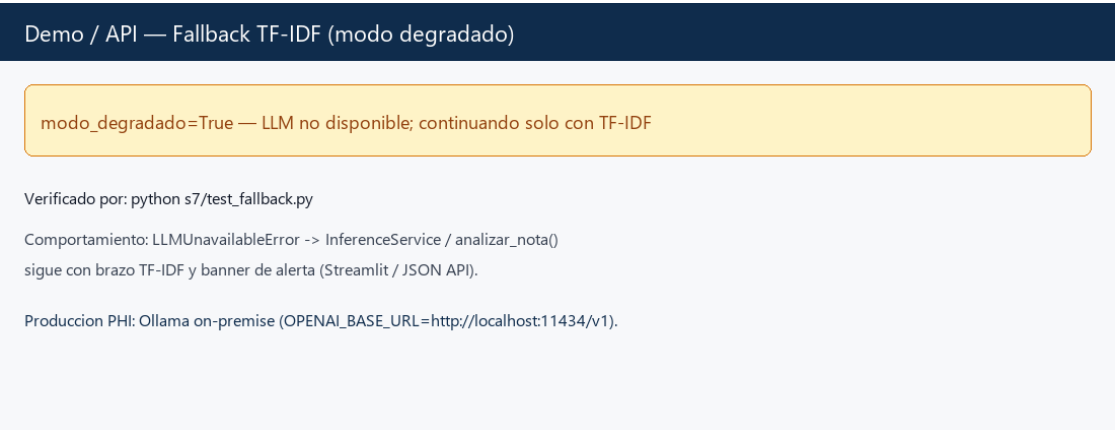


Figura 7. Fallback TF-IDF (modo degradado).

7. Pitch (escrito)

Problema: las inconsistencias clinicas son contradicciones semanticas (p. ej. alergia a penicilina + prescripcion de amoxicilina), invisibles para TF-IDF a nivel nota.

Solucion: segmentar la nota, puntuar cada oracion y resaltar la mas sospechosa; opcionalmente enriquecer con LLM+RAG anclado en guias clinicas.

Evidencia: ROC-AUC 0.949 y localizacion 84.6 % sobre MEDEC; demo Streamlit o frontend React + API FastAPI en minutos.

Valor: apoyo a auditoria clinica y seguridad del paciente; despliegue recomendado en cascada TF-IDF local -> LLM solo en candidatas; Ollama para PHI; fallback transparente.

Mensaje clave: el prototipo no diagnostica ni prescribe - prioriza oraciones sospechosas para revision humana, con trazabilidad (scores, RAG, auditoria).

8. Reflexion y retroalimentacion

Feedback del curso integrado: reportar AUPRC ademas de F1 (prevalencia 4.5 %); diagnosticar sesgo EN-ES; documentar riesgos PHI antes de piloto CITIMED.

Leccion S6: datos sinteticos generaron metricas enganosas (AUC 0.845) - validacion solo sobre MEDEC real.

Leccion S7: el valor clinico esta en localizar la oracion, no en clasificar la nota entera.

Correccion documental S10: el borrador S8 listaba capacidades ya implementadas como pendientes; este informe se reescribio desde el repositorio (rutas, metricas JSON, FastAPI/React) y separa claramente trabajo futuro (LangChain, Docker, DVC, MLflow).

Mejora aplicada post-feedback: eval_tfidf_idioma.py (stop words espanol) eleva test AUC a ~0.959 vs 0.949 english-only - evidencia en s10/evidencias/eval_tfidf_idioma.json.

9. Limitaciones y trabajo futuro

- LLM evaluado principalmente en mock - conclusiones semanticas requieren API/Ollama real.
- Subset tripartita (500 oraciones) - metricas completas en test requieren mas compute.
- Corpus CITIMED anonimizado pendiente (organizacional + etica).
- Cascada TF-IDF->LLM documentada pero no codificada en inferencia.py.
- Docker Compose para empaquetar API + frontend: plan futuro (FastAPI ya implementada).
- LangChain, DVC, MLflow: plan futuro, no entregados en S10.

10. Anexo tecnico

Estructura del repositorio:

```
MIA_Deteccion_HC/  
??? s6/      # TF-IDF ajustado + BITACORA  
??? s7/      # LLM, RAG, evaluaciones, inferencia  
??? api/     # FastAPI - POST /generar, GET /health  
??? frontend/ # React + Vite (cliente HTTP)  
??? demo/    # Streamlit (demo alternativa)  
??? s8/docs/  # Borrador historico S8  
??? s10/     # Entrega S10 (evidencias + PDF)  
??? salidas_ajuste/ # modelo_ajustado.joblib (generado)  
??? salidas_s7/  # metricas eval, FAISS, cache (generado)
```

Comandos de reproduccion (desde raiz, venv activo, MEDEC en medec_try/MEDEC-MS/):

```
python s6/modelo_ajustado.py  
python s7/analisis_por_tipo.py  
python s7/eval_tripartita.py --mock-llm --max-oraciones 500
```

```
python s7/eval_idioma.py --mock-llm --subset 200
python s7/eval_tfidf_idioma.py
python s7/test_fallback.py
python s10/capture_demo.py
python s10/organize_evidencias.py
python s10/generate_informe.py
uvicorn api.main:app --reload --port 8000
cd frontend && npm install && npm run dev
streamlit run demo/app.py
```

Anexo A - Evidencias cuantitativas y capturas en s10/evidencias/ del repositorio.

11. Referencias

Ben Abacha, A. B., et al. (2025). MEDEC: A Benchmark for Medical Error Detection and Correction in Clinical Notes.

Repositorio del proyecto: https://github.com/marcoz1691/MIA_Deteccion_HC

Documentación complementaria: s7/docs/informe_produccion.md, s7/docs/reporte_interpretabilidad_etica.md, s6/BITACORA.md, s10/README.md.